# Image Processing Filters

## Unit 2 — Performance Analysis

**Raúl Cetina**

**Daniel Gómez**

**Christopher Quiñones**

High-Performance Computing

March 2026

Repository: https://github.com/ezraidenn/Images-Processing

# Contents

**1 Problem Description** **2**
1.1 Gaussian Filter (Blurring / Noise Reduction) . . . . . 2
1.2 Sobel Filter (Edge Detection) . . . . . 2
1.3 Median Filter (Noise Reduction) . . . . . 2

**2 Implementation Details** **3**
2.1 Pure Python . . . . . 3
2.2 NumPy . . . . . 3
2.3 NumPy + Cython . . . . . 3

**3 Performance Analysis** **4**
3.1 Execution Times . . . . . 4
3.2 Performance Chart . . . . . 5
3.3 Key Observations . . . . . 5

**4 Visual Results** **5**
4.1 Gaussian Filter . . . . . 6
4.2 Sobel Filter . . . . . 6
4.3 Median Filter . . . . . 7

**5 Discussion and Conclusions** **7**
5.1 Trade-offs . . . . . 7
5.2 Insights . . . . . 7
5.3 Conclusion . . . . . 8

# Problem Description

This project implements three fundamental image processing filters and evaluates their performance across three distinct computational approaches. The objective is to understand how algorithmic optimization and compiled code affect execution speed for pixel-level operations.

## Gaussian Filter (Blurring / Noise Reduction)

The Gaussian filter smooths an image by replacing each pixel with a weighted average of its neighbors, computed using a Gaussian kernel. The standard $3 \times 3$ kernel is:

$$G = \frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} \tag{1}$$

The kernel assigns higher weight to the center pixel and decreasing weights to surrounding pixels, producing a smooth blurring effect. This filter is widely used to reduce Gaussian noise before subsequent processing.

## Sobel Filter (Edge Detection)

The Sobel filter detects edges by computing the gradient of the image intensity in both the horizontal ($x$) and vertical ($y$) directions using two separate kernels:

$$S_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} \qquad S_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix} \tag{2}$$

The gradient magnitude is then computed as:

$$G = \sqrt{S_x^2 + S_y^2} \tag{3}$$

This highlights regions of rapid intensity change, effectively detecting the edges of objects.

## Median Filter (Noise Reduction)

The median filter is a nonlinear filter that replaces each pixel with the *median* value of its $3 \times 3$ neighborhood. For example, given:

$$\begin{pmatrix} 10 & 50 & 200 \\ 30 & 25 & 70 \\ 20 & 80 & 90 \end{pmatrix}$$

the sorted values are $\{10, 20, 25, 30, \mathbf{50}, 70, 80, 90, 200\}$, and the center pixel would be replaced by 50. This method is particularly effective for removing salt-and-pepper noise while preserving edge sharpness.

# Implementation Details

Each filter was implemented using three progressively optimized approaches.

## Pure Python

The pure Python implementation uses only built-in data structures (nested lists) and basic arithmetic. No external libraries are employed for the computation itself. This approach serves as the baseline:

```
# Gaussian filter excerpt (Pure Python)
for i in range(1, rows - 1):
    for j in range(1, cols - 1):
        pixel_sum = 0
        for ki in range(-1, 2):
            for kj in range(-1, 2):
                pixel_sum += image[i+ki][j+kj] * kernel[ki+1][kj+1]
        output[i][j] = pixel_sum // 16
```

The nested loop structure results in $\mathcal{O}(n \times m \times k^2)$ complexity with pure Python overhead per iteration.

## NumPy

The NumPy implementation replaces explicit loops with vectorized array slicing. Instead of iterating over every pixel, we leverage NumPy's C-backed operations:

```
# Gaussian filter excerpt (NumPy)
for ki in range(3):
    for kj in range(3):
        output[1:rows-1, 1:cols-1] += (
            image[ki:rows-2+ki, kj:cols-2+kj].astype(np.float64)
            * kernel[ki, kj]
        )
```

The inner loop runs only 9 iterations (kernel size), while each iteration processes the entire image in a single vectorized operation. For the median filter, all 9 neighbors are stacked and `np.median` computes the result along the stack axis.

## NumPy + Cython

The Cython implementation combines NumPy arrays with C-level typed access via Cython's `ndarray` buffer protocol. Key optimizations include:

- **Typed variables**: All loop indices and accumulators are declared as C `int` or `double`.
- **Bounds-check disabled**: The directive `boundscheck=False` eliminates array bounds checking overhead.
- **Inline kernel**: The convolution kernel is stored in a C array, avoiding Python object overhead.
- **Insertion sort**: For the median filter, a C-level insertion sort on 9 elements replaces Python's sort.

```
# Gaussian filter excerpt (Cython)
cdef int kernel[3][3]
kernel[0][0]=1; kernel[0][1]=2; kernel[0][2]=1
# ... (initialized once)
for i in range(1, rows - 1):
    for j in range(1, cols - 1):
        pixel_sum = (image[i-1,j-1]*kernel[0][0] + ...)
        output[i, j] = <cnp.uint8_t>(pixel_sum // 16)
```

The module is compiled using `setup.py` with `Cython.Build.cythonize`.

# Performance Analysis

All benchmarks were run on a $512 \times 512$ grayscale test image, averaged over 3 runs using `time.perf_counter()`.

## Execution Times

| Filter | Approach | Avg. Time (s) | Speedup |
|---|---|---|---|
| Gaussian | Pure Python | 0.345418 | 1.00× |
| | NumPy | 0.009329 | 37.03× |
| | NumPy + Cython | 0.000564 | 612.44× |
| Sobel | Pure Python | 0.471307 | 1.00× |
| | NumPy | 0.017792 | 26.49× |
| | NumPy + Cython | 0.002245 | 209.94× |
| Median | Pure Python | 0.240595 | 1.00× |
| | NumPy | 0.008140 | 29.56× |
| | NumPy + Cython | 0.002221 | 108.33× |

Table 1: Performance comparison across filter implementations.

## Performance Chart

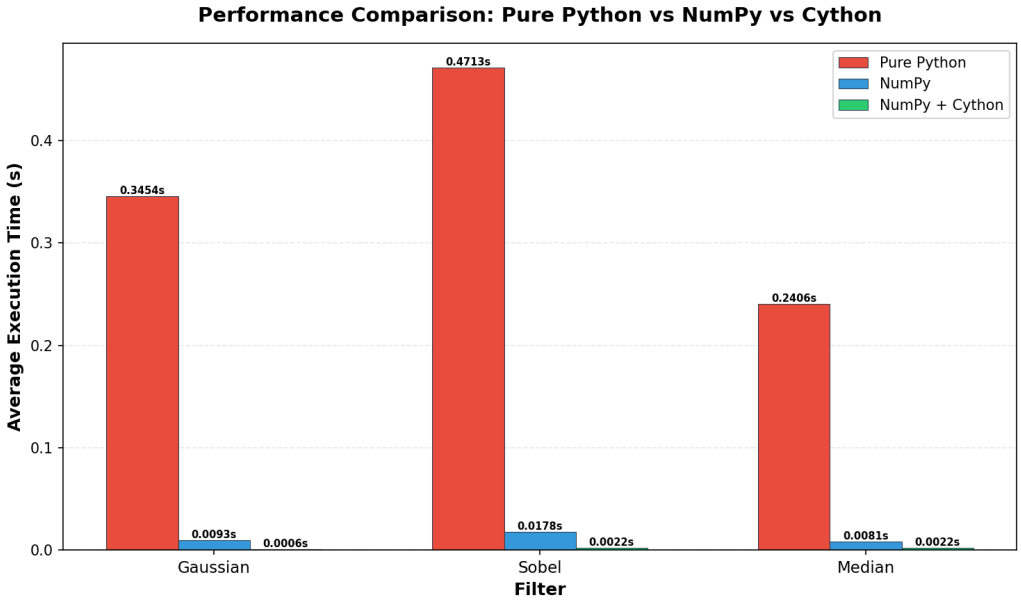


Figure 1: Bar chart comparing average execution times across all filter–approach combinations.

## Key Observations

1. **Pure Python is orders of magnitude slower.** The four nested loops for convolution result in heavy Python interpreter overhead per pixel.
2. **NumPy achieves 26–37× speedup** by replacing pixel-level loops with vectorized C operations. The median filter benefits from `np.median` operating on stacked arrays.
3. **Cython delivers 108–612× speedup** by compiling to C, eliminating Python object overhead, and performing direct memory access through typed buffers. The Gaussian filter sees the largest benefit because its computation is arithmetic-intensive with uniform memory access patterns.
4. **Sobel is the most computationally expensive** in Pure Python because it requires two convolutions (X and Y kernels) plus a square root per pixel.

# Visual Results

The following figures show the original image alongside the result of each filter applied using all three approaches. Visually, the filter outputs are identical across approaches—only execution time differs.

## Gaussian Filter

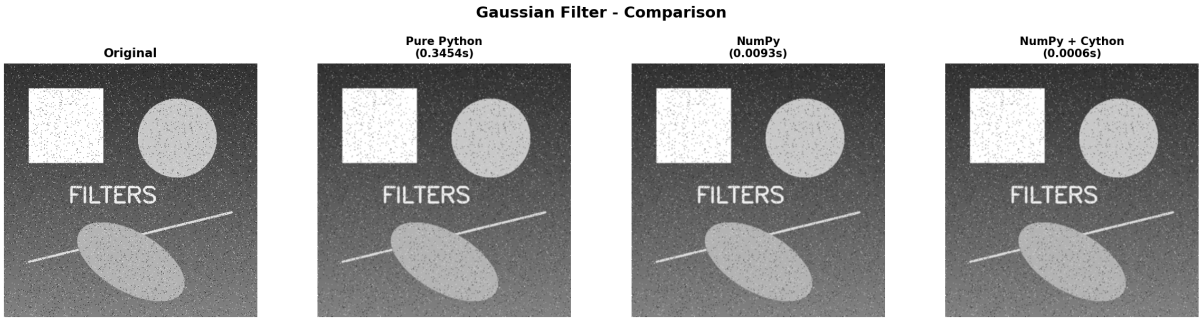


Figure 2: Gaussian filter results: Original (left) vs. Pure Python, NumPy, and Cython outputs.

The Gaussian filter produces a smooth, blurred version of the original image. The salt-and-pepper noise present in the original is visibly reduced, and edges become softer.

## Sobel Filter

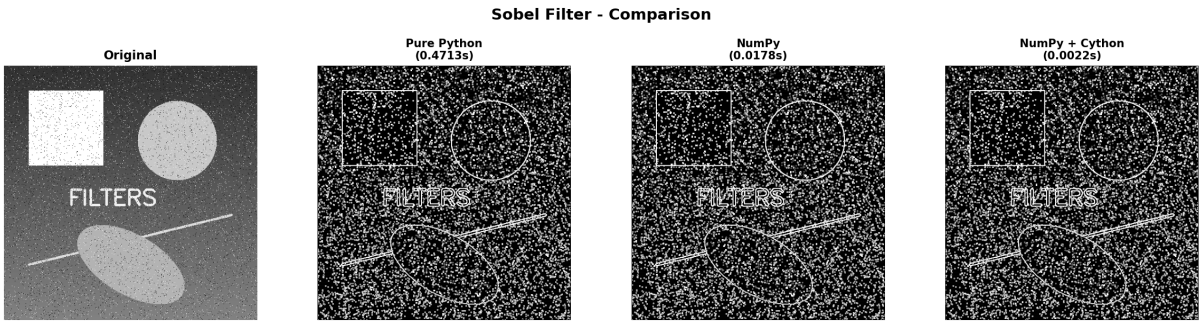


Figure 3: Sobel filter results: Original (left) vs. Pure Python, NumPy, and Cython outputs.

The Sobel filter clearly highlights the edges of all geometric shapes in the test image. The text “FILTERS”, rectangle, circle, ellipse, and diagonal line are all distinctly outlined against the dark background.

## Median Filter

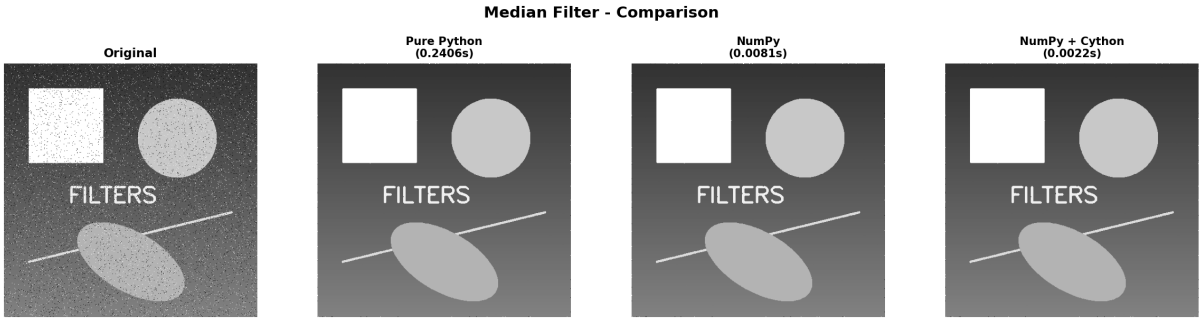


Figure 4: Median filter results: Original (left) vs. Pure Python, NumPy, and Cython outputs.

The median filter effectively removes the salt-and-pepper noise while preserving the sharp edges of the shapes. Unlike the Gaussian filter, edges remain crisp—this is the key advantage of median filtering for impulse noise.

# Discussion and Conclusions

## Trade-offs

| Approach | Speed | Ease of Impl. | Setup Complexity |
|---|---|---|---|
| Pure Python | Slowest | Easiest | None |
| NumPy | Fast | Easy | `pip install numpy` |
| NumPy + Cython | Fastest | Moderate | C compiler required |

Table 2: Trade-off summary between approaches.

## Insights

- **Pure Python** is adequate for prototyping and small images but becomes impractical for real-time or large-scale image processing. Its primary value is educational: the explicit loops make the algorithm transparent.
- **NumPy** strikes the best balance between performance and developer productivity. With a single `pip install`, it achieves 26–37× speedup with minimal code complexity. For most practical applications, NumPy is the recommended choice.
- **Cython** is justified when extreme performance is critical, such as processing video streams or very high-resolution images. The 100–600× speedup comes at the cost of needing a C compiler and writing slightly more verbose code with type annotations.

- The **diminishing returns** from NumPy to Cython depend on the filter type. Arithmetic-heavy operations (like Gaussian convolution) benefit the most from Cython's compiled arithmetic. The median filter, which relies on sorting, shows a smaller (but still substantial) improvement because the bottleneck shifts from Python overhead to algorithmic complexity.

## Conclusion

This project demonstrates that computational optimization can dramatically accelerate image processing operations. By transitioning from Pure Python to NumPy and then to Cython, we achieved speedups of up to $612\times$ while maintaining identical output quality. The choice of approach should be guided by the application's performance requirements and development constraints.